IMEN574 / IDSC574 · PROGRAMMING FOR AX

5

Python for Data Work

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 **Programming with AI**

Lectures 2–4

02 **Python and Classical Machine Learning**

Lectures 5–13

03 **Deep Learning and Foundation Models**

Lectures 14–22

Outline: Python for Data Work

01 **Read Values and State**

Trace the data through expressions

02 **Build Testable Functions**

Make dependencies explicit

03 **Use Model Interfaces**

Separate state from behavior

01 · Read Values and State

01 **Read Values and State**
Trace the data through expressions

02 **Build Testable Functions**
Make dependencies explicit

03 **Use Model Interfaces**
Separate state from behavior

Why Read the Function

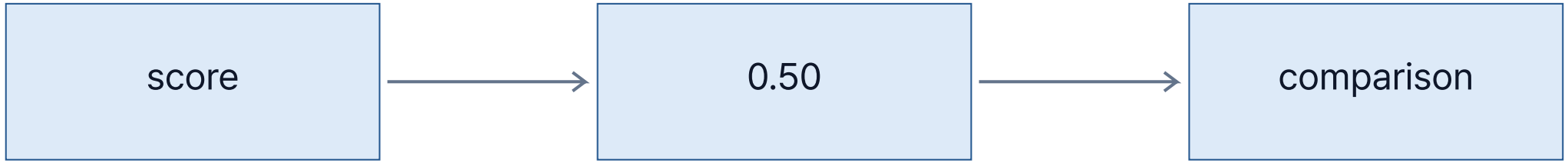
A one-character change can alter a decision

CONTRACT	CANDIDATE
positive iff score >= threshold	positive iff score > threshold
classify([.5]) → [1]	classify([.5]) → [0]

The boundary lives inside one comparison

Names and Values

Assignment binds a name to an object



A name

A float object

score >= .50 is True

A name refers to a value used by an expression

A Data Contract

Types express different kinds of input

Name	Value	Type
score	0.5	float
label	1	int
valid	True	bool
scores	[.2,.8]	list

The representation is part of the input contract

Conversions at the Boundary

Parse text before applying a numeric rule

```
raw = ["0.20", "0.50", "0.80"]
scores = [float(s) for s in raw]
print(scores)
print([int(s >= .5) for s in scores])
```

```
[0.2, 0.5, 0.8]
[0, 1, 1]
```

Parsing turns text into values the rule can compare

Selection and Slicing

A slice excludes its stop index

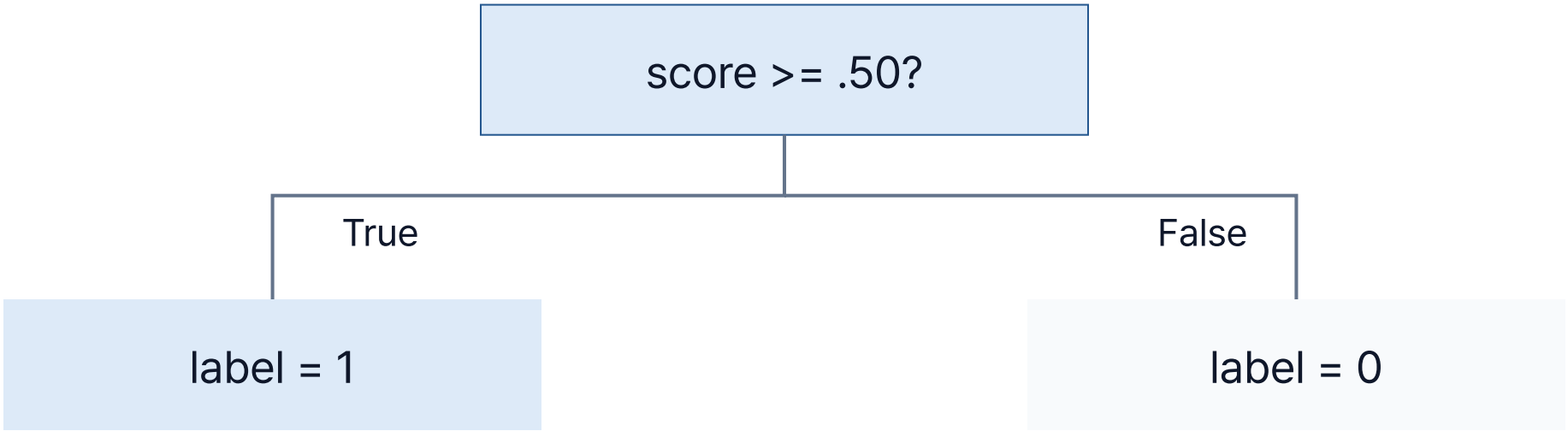
	Index	Score
Keep	0	.20
Keep	1	.50
Stop	2	.80
	3	.95

```
scores[0:2] = [0.20, 0.50]
```

The stop index is outside the selected range

Control Flow

Choose the path from the input



A condition selects one of two outcomes

Iteration and Order

A loop visits the sequence in order

Position	Input	Output
0	.20	0
1	.50	1
2	.80	1

Ordered iteration preserves the row correspondence

Comprehensions

Build one output from each input

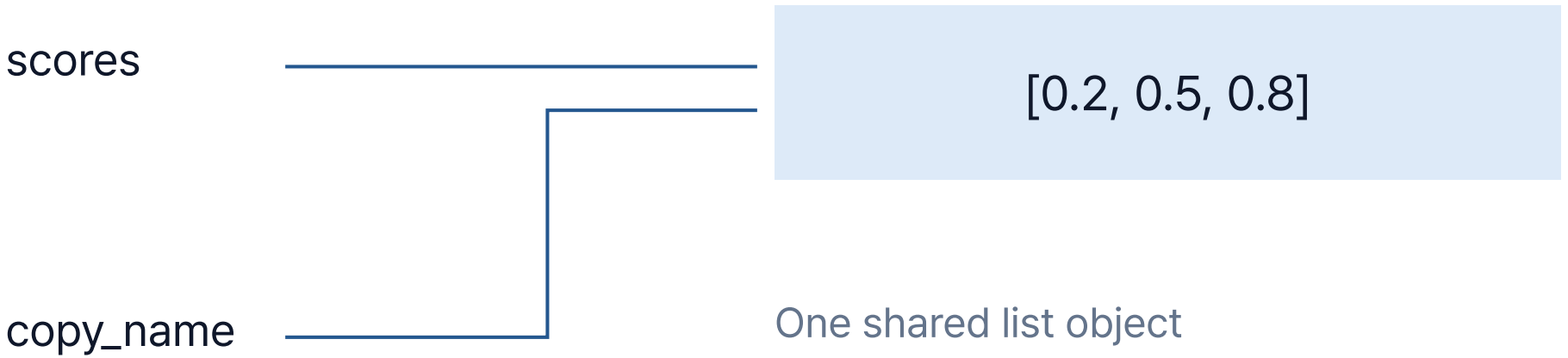
```
scores = [.2, .5, .8]
labels = []
for score in scores:
    labels.append(int(score >= .5))
short = [int(s >= .5) for s in scores]
print(labels == short)
```

```
True
```

The short form preserves the loop meaning

Mutable State

Two names can refer to the same list



Two references do not create two lists

https://docs.python.org/3/tutorial/classes.html

Caveat: Shared Mutation

A helper can change the caller input

```
scores = [.2, .5, .8]
other = scores
other.append(.95)
print(scores)
print(other is scores)
```

```
[0.2, 0.5, 0.8, 0.95]
True
```

A mutation is visible through every alias

A Deliberate Copy

Copy when independent edits are required

```
scores = [.2, .5, .8]
other = scores.copy()
other.append(.95)
print(scores)
print(other)
```

```
[0.2, 0.5, 0.8]
[0.2, 0.5, 0.8, 0.95]
```

The copied outer list can change independently

Demo: Shared Python State

Compute the result from the input before changing it

Run the code; predict both lists Replace alias assignment with scores.copy(); run again

Python

```
scores = [0.2, 0.5, 0.8]
alias = scores
alias.append(0.95)
copy = scores.copy()
copy.append(0.99)
print("scores", scores)
print("alias is scores", alias is scores)
print("copy", copy)
```

Run Python Reset

```
scores [0.2, 0.5, 0.8, 0.95]
alias is scores True
copy [0.2, 0.5, 0.8, 0.95, 0.99]
```

Answer: Shared Python State

The same input produces these calculated values

Input / quantity	Result
alias = scores	scores [0.2,0.5,0.8,0.95]
alias = scores.copy()	scores [0.2,0.5,0.8]

Caveat: Shared Python State

A shallow copy still shares nested objects

Condition	Consequence
nested=[[1]]; other=nested.copy()	other[0].append(9) also changes nested

02 · Build Testable Functions

01 Read Values and State

Trace the data through expressions

02 **Build Testable Functions**

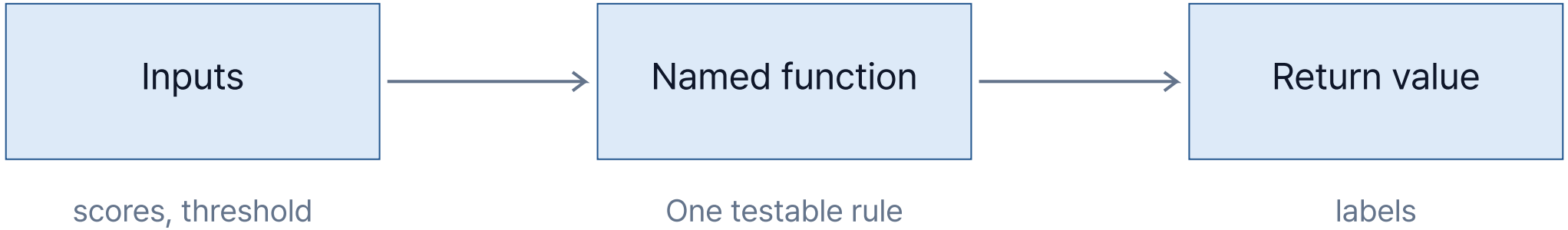
Make dependencies explicit

03 Use Model Interfaces

Separate state from behavior

Why Functions

A repeated rule needs one definition



One definition gives the rule a stable boundary

Parameters and Return

Make dependencies visible in the signature

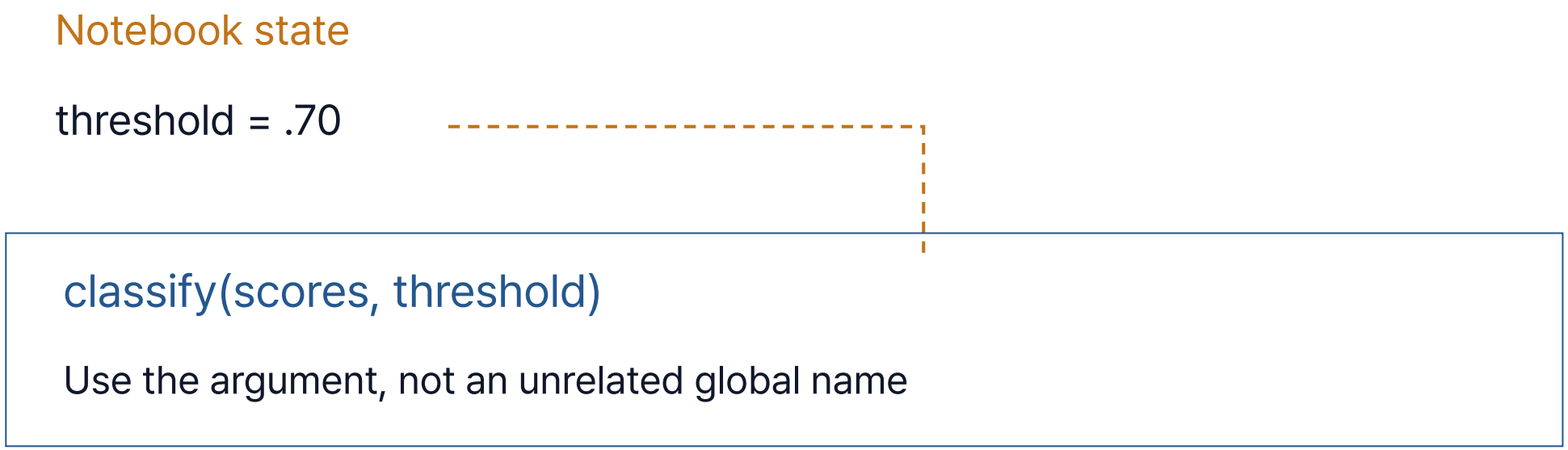
```
def classify(scores, threshold=.5):
    return [int(s >= threshold) for s in
scores]
print(classify([.2, .5, .8], threshold=.7))
```

```
[0, 0, 1]
```

An explicit parameter exposes the changing assumption

Local and Global Names

A function should not depend on hidden notebook state

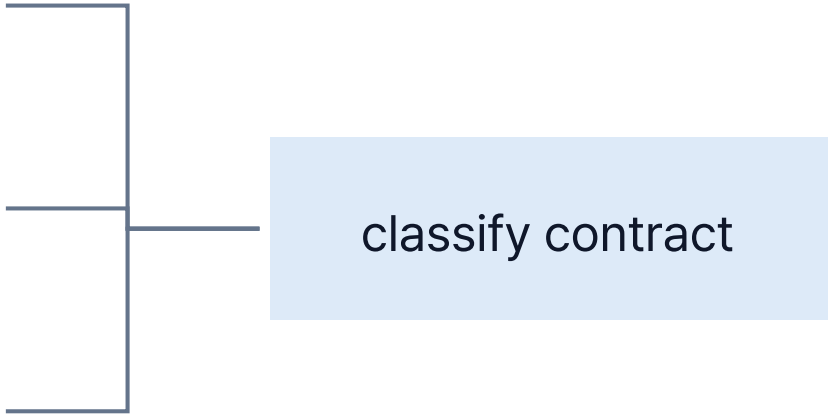


An explicit argument replaces a hidden dependency

A Function Contract

Document meaning as well as syntax

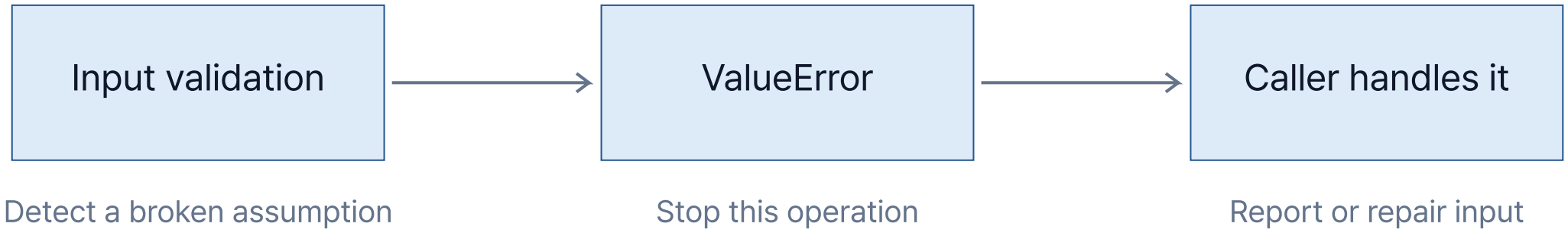
Input	Finite scores in [0,1]
Rule	score >= threshold
Output	One integer per input row



Input assumptions and output shape constrain the function

Exceptions

An invalid input needs a visible failure



Invalid input follows a different path from a negative label

A Narrow Exception Handler

Catch the failure you can explain

```
def checked_score(raw):
    try:
        value = float(raw)
    except ValueError:
        return None
    return value
print([checked_score(x) for x in [".5",
"bad"]])
```

```
[0.5, None]
```

The handler explains one known conversion failure

Caveat: Mutable Defaults

A default list can survive between calls

```
def add_score(value, history=[]):
    history.append(value)
    return history
print(add_score(.2))
print(add_score(.8))
```

```
[0.2]
[0.2, 0.8]
```

The second call inherits the first call state

A Fresh Default

Create per-call state inside the function

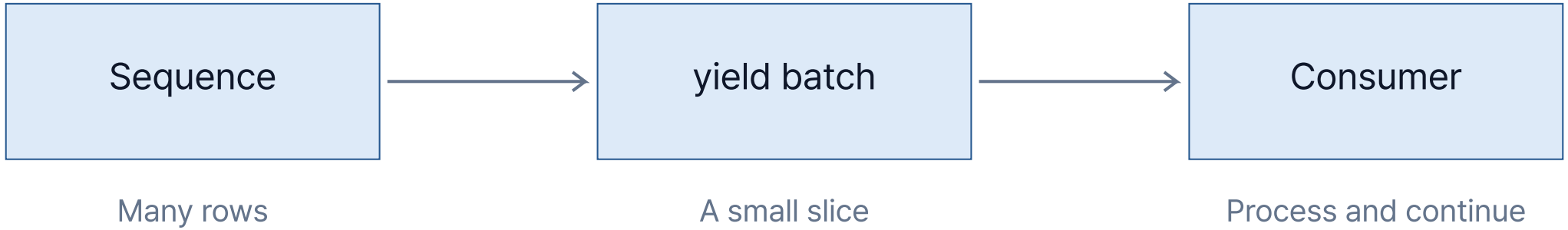
```
def add_score(value, history=None):
    if history is None:
        history = []
    return history + [value]
print(add_score(.2))
print(add_score(.8))
```

```
[0.2]
[0.8]
```

Each default call starts with independent state

Generators and Batches

Yield only the next piece of work



A generator exposes one batch at a time

A Batch Generator

Keep the last partial batch

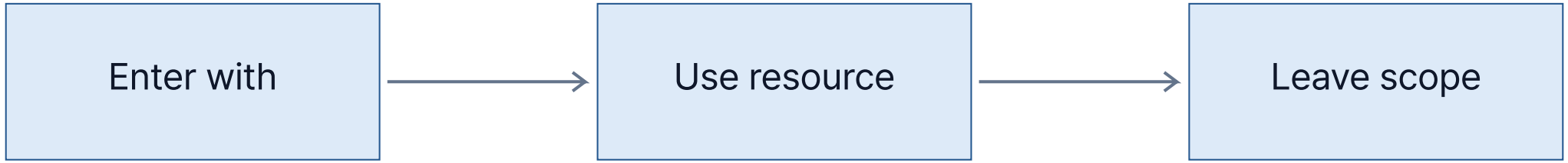
```
def batches(values, size):
    for start in range(0, len(values),
size):
        yield values[start:start + size]
print(list(batches([1, 2, 3, 4, 5], 2)))
```

```
[[1, 2], [3, 4], [5]]
```

Five rows form two full batches and one partial batch

Context Managers

Release a resource after its scope ends



Open a handle

Read the input

Close even on an error

The scope controls the resource lifetime

03 · Use Model Interfaces

01 Read Values and State

Trace the data through expressions

02 Build Testable Functions

Make dependencies explicit

03 **Use Model Interfaces**

Separate state from behavior

Why an Object Interface

Stateful models need a consistent protocol



A model object separates learning from use

Class and Instance

Each instance owns its learned state

Instance	State
model_a	mean = 2
model_b	mean = 7

One class can create models with different states

A Minimal Estimator

Implement the protocol before adding abstractions

```
class MeanModel:
    def fit(self, y):
        if not y: raise ValueError("empty target")
        self.mean_ = sum(y) / len(y)
        return self
    def predict(self, n):
        return [self.mean_] * n
```

The interface exposes the learned baseline

Demo: Independent Model Instances

Fit two instances without sharing their learned state

Run two independent fitted objects Change only b training targets to [10,14]

Python: MeanModel is defined on the previous slide

```
a = MeanModel().fit([1, 2, 3])
b = MeanModel().fit([6, 8])
print(a.predict(2), b.predict(2))
```

Run Python Reset

```
[2.0, 2.0] [7.0, 7.0]
```

Answer: Independent Model Instances

The same input produces these calculated values

Input / quantity	Result
a fit [1,2,3]	predict(2) = [2.0,2.0]
b fit [6,8] → [10,14]	[7.0,7.0] → [12.0,12.0]

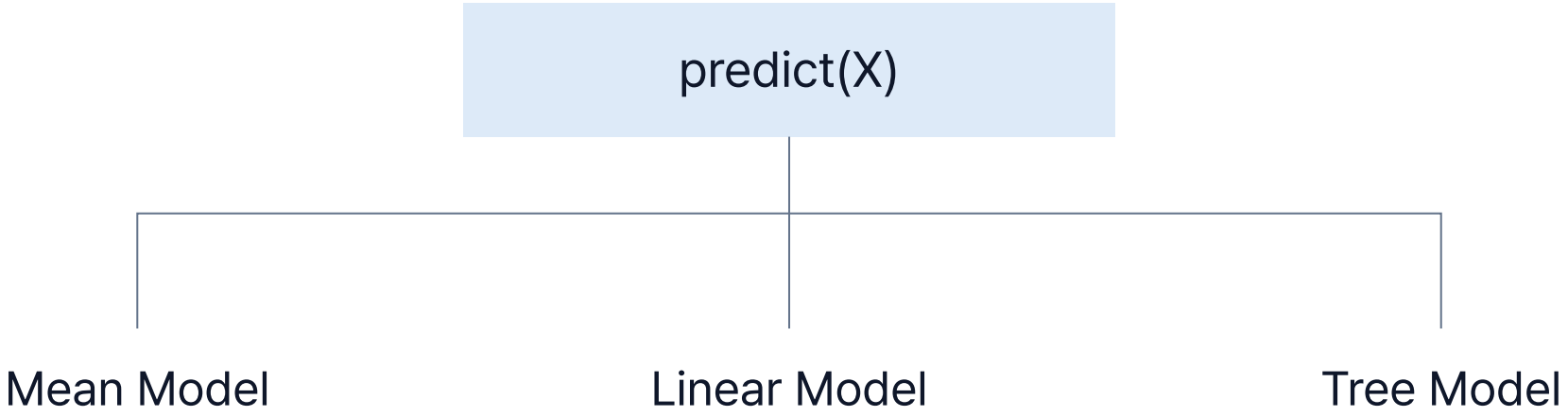
Caveat: Independent Model Instances

The estimator must be fitted before prediction

Condition	Consequence
MeanModel().predict(1)	AttributeError: no fitted mean_
MeanModel().fit([])	ValueError: empty target

Polymorphism

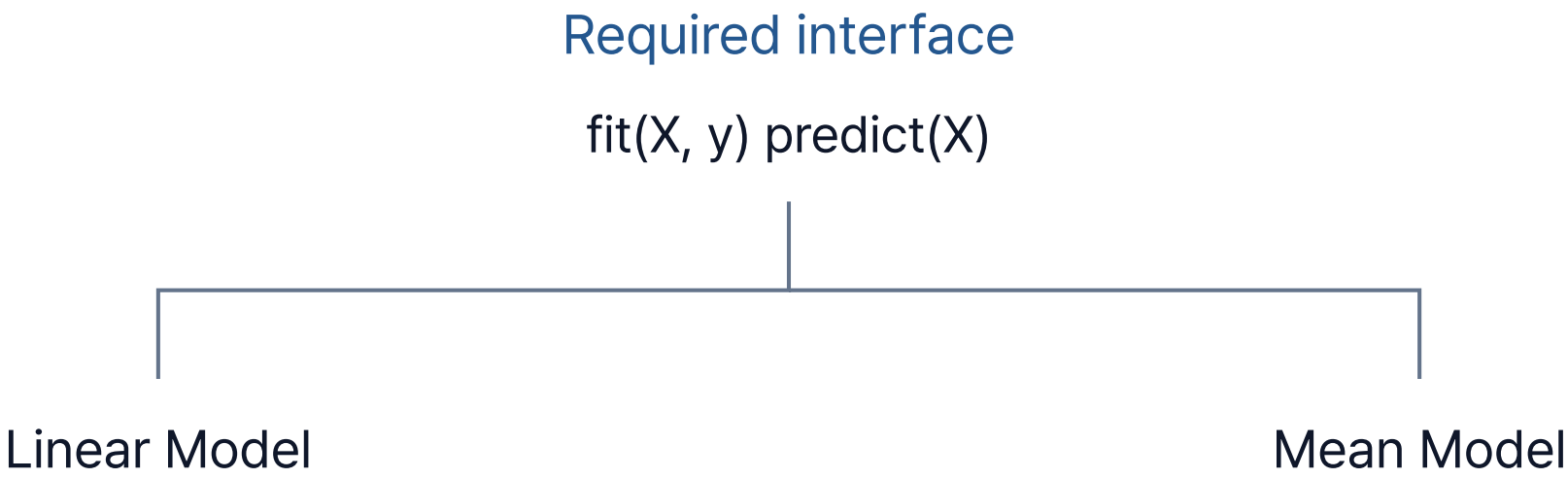
Different models can share one calling pattern



A shared protocol supports model comparison

Abstract Interfaces

An abstract method names a required behavior



Each implementation supplies both behaviors

The interface defines shared behavior for alternative implementations

Python Data Protocols

Special methods connect objects to familiar syntax

Expression	Protocol Method
len(data)	__len__
data[i]	__getitem__
model(X)	__call__
repr(model)	__repr__

Familiar syntax calls a documented object protocol

Types and Dataclasses

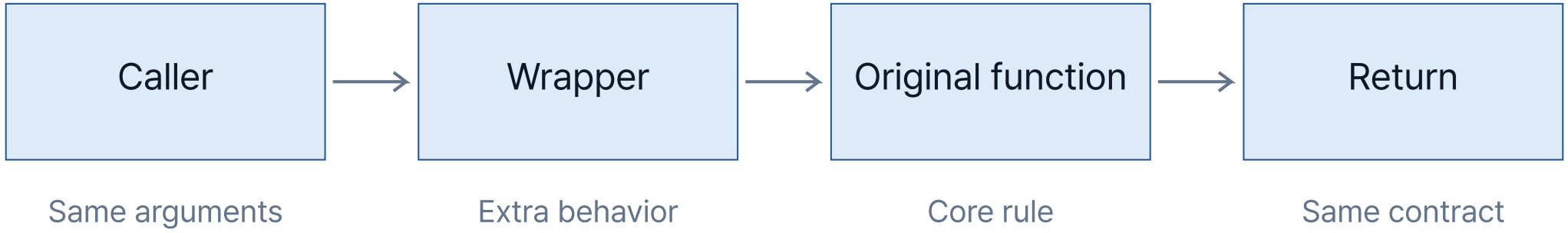
Describe fields without hiding runtime checks

```
from dataclasses import dataclass
@dataclass
class Config:
    threshold: float = .5
    seed: int = 7
config = Config(threshold=.7)
print(config)
```

A data declaration is not input validation

Decorators

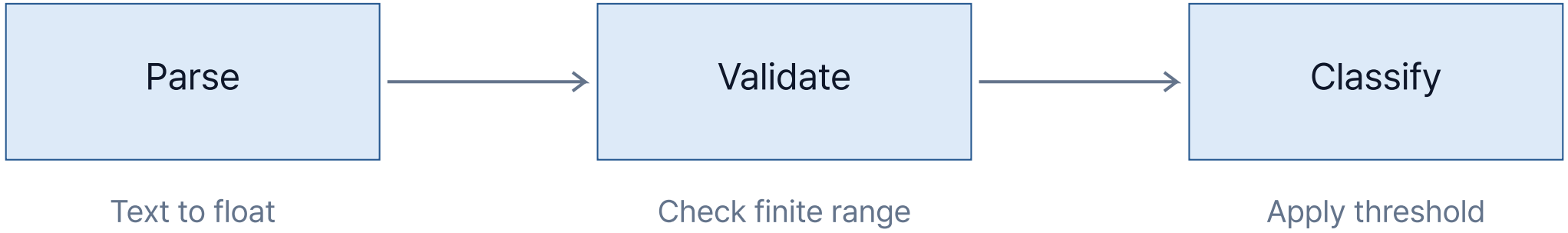
Wrap a function without changing its call site



A wrapper adds behavior around the same function

Function Composition

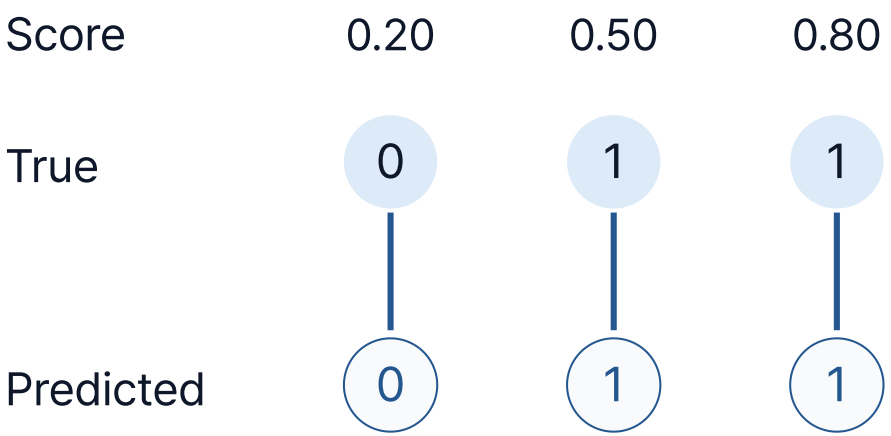
Keep small transformations independently testable



Composition exposes three independent contracts

A Readable Repair

Explain a generated function in terms of its contract




```
classify(scores, threshold=.5) returns one label per input
```

Code reading traces the contract to each returned value

Takeaway: Python for Data Work

01 Trace names, values and mutations

02 Expose inputs and failures in functions

03 Use a stable interface for learned state